

PONTIFICIA UNIVERSIDAD JAVERIANA

Facultad de Ingeniería

Introducción a la Inteligencia Artificial

Taller EDA – Fase 2

Evaluación de Arquitecturas de Redes Neuronales Feed-Forward

para la Clasificación de Ingresos

Dataset: Adult (Census Income) – UCI Repository

Docente:

Ing. Julio Omar Palacio Niño, M.Sc.

palacio_julio@javeriana.edu.co

Objetivo:

Diseñar, implementar y comparar arquitecturas de redes neuronales feed-forward para predecir si una persona gana más o menos de 50.000 USD al año, empleando los datos preprocesados de la Fase 1.

Bogotá D.C., 2026

TABLA DE CONTENIDO

1. Introducción	3
2. Descripción del dataset y conexión con la Fase 1	3
2.1. Características del conjunto de datos	3
2.2. Resultados de la Fase 1 que condicionan el diseño	3
2.3. Conjuntos de trabajo	4
3. Entorno de trabajo y herramientas	4
4. Especificaciones técnicas de los modelos	5
4.1. Condiciones generales de diseño	5
4.2. Modelo A: Perceptrón	5
4.3. Modelo B: Red neuronal con una capa oculta	6
4.4. Modelo C: Red neuronal con dos capas ocultas	7
5. Análisis de resultados y evaluación	7
5.1. Modelo A: Perceptrón	7
5.1.1. Matriz de confusión	7
5.1.2. Métricas de desempeño	8
5.2. Modelo B: Red neuronal con una capa oculta	9
5.2.1. Matriz de confusión	9
5.2.2. Métricas de desempeño	9
5.3. Modelo C: Red neuronal con dos capas ocultas	10
5.3.1. Matriz de confusión	10
5.3.2. Métricas de desempeño	10
6. Manejo del desbalanceo de clases	10

7. Análisis comparativo	11
7.1. Tabla consolidada de métricas	11
7.2. Discusión técnica	12
7.2.1. Modelo A – Perceptrón	12
7.2.2. Modelo B – Red neuronal con una capa oculta	12
7.2.3. Modelo C – Red neuronal con dos capas ocultas	12
7.3. Justificación de la arquitectura ganadora	12
8. Conclusiones	13
9. Referencias	14

1. Introducción

Este informe corresponde a la entrega de la Fase 2 del Proyecto Final de Introducción a la Inteligencia Artificial. El objetivo es diseñar, implementar y evaluar tres arquitecturas de redes neuronales feed-forward para el problema de clasificación binaria sobre el dataset *Adult* del Repositorio UCI.

La tarea consiste en predecir si una persona gana más o menos de 50.000 dólares anuales, a partir de variables sociodemográficas del censo de los Estados Unidos de 1994. Los datos utilizados provienen directamente del preprocesamiento realizado en la Fase 1, sin modificaciones adicionales. Esta continuidad es fundamental: las decisiones tomadas en la Fase 1 (número de variables, tratamiento del desbalanceo, normalización y partición) determinan directamente las condiciones de diseño de los modelos evaluados en esta fase.

Se implementaron tres arquitecturas: el Perceptrón (Modelo A), una red neuronal con una capa oculta (Modelo B) y una red neuronal con dos capas ocultas (Modelo C). Para cada una se construyó la matriz de confusión y se calcularon las métricas de exactitud, precisión, *recall* y F1-score. El informe concluye con un análisis comparativo que justifica la arquitectura de mejor desempeño para este problema específico.

2. Descripción del dataset y conexión con la Fase 1

2.1. Características del conjunto de datos

El dataset *Adult* contiene 48.842 registros del censo de los Estados Unidos de 1994. La variable objetivo es `income`: 0 para ingresos iguales o menores a 50K USD y 1 para ingresos superiores. Este es un problema de clasificación binaria con desbalanceo inherente entre clases.

2.2. Resultados de la Fase 1 que condicionan el diseño

El preprocesamiento realizado en la Fase 1 produjo los conjuntos de datos que se emplean directamente en esta fase. Los resultados de esa etapa que inciden directamente sobre el diseño de los modelos son los siguientes:

1. **Desbalanceo de clases:** El 76,1 % de los registros corresponde a la clase $\leq 50K$ y

solo el 23,9 % a la clase >50K. Este desbalanceo obliga a incorporar estrategias de compensación durante el entrenamiento (uso de `class_weight='balanced'`) y a priorizar métricas como el F1-score y el *recall* de la clase minoritaria por encima de la exactitud bruta.

2. **Dimensionalidad del espacio de entrada (82 *features*):** El preprocesamiento transformó las 14 columnas originales en 82 variables numéricas o binarias, como resultado de: la eliminación de la columna redundante `education`, la imputación de nulos con la moda, la codificación *one-hot* con `drop_first=True` (para evitar multicolinealidad perfecta) y la normalización con `StandardScaler`. Este número determina directamente la cantidad de neuronas de la capa oculta del Modelo B (igual al número de variables de entrada).
3. **Partición estratificada 80/20:** Se empleó `stratify=y` para conservar la proporción de clases 76/24 tanto en el conjunto de entrenamiento como en el de prueba, con `random_state=42` para garantizar reproducibilidad.
4. **Normalización con `StandardScaler`:** Los modelos basados en gradiente (B y C) son sensibles a la escala de las variables. La normalización aplicada en la Fase 1 es condición necesaria para la convergencia adecuada de estos modelos.

2.3. Conjuntos de trabajo

La tabla 1 resume las dimensiones de los conjuntos utilizados.

Tabla 1: Dimensiones de los conjuntos de entrenamiento y prueba

Conjunto	Filas	Columnas	Clase mayoritaria
Entrenamiento (<code>X_train</code>)	39.073	82	76,1 %
Prueba (<code>X_test</code>)	9.769	82	76,1 %

3. Entorno de trabajo y herramientas

El desarrollo de esta fase se realizó en Python, ejecutado sobre Google Colab para garantizar la reproducibilidad del experimento en un entorno en la nube sin dependencias locales. Las

librerías especializadas utilizadas son:

- **Scikit-learn:** Para la implementación del Perceptrón (Modelo A), el cálculo de métricas (`classification_report`, `confusion_matrix`) y la ponderación de clases con `compute_class_weight`.
- **TensorFlow / Keras:** Para la construcción, compilación y entrenamiento de los Modelos B y C mediante la API secuencial de Keras (`tf.keras.Sequential`).
- **NumPy / Pandas:** Para la manipulación de los datos preprocesados en la Fase 1.
- **Matplotlib / Seaborn:** Para la visualización de matrices de confusión y curvas de entrenamiento.

4. Especificaciones técnicas de los modelos

4.1. Condiciones generales de diseño

Las condiciones de diseño para esta fase son:

- Todas las neuronas, en todos los modelos, deben incluir obligatoriamente el término de sesgo (*bias*).
- Para los Modelos B y C, la función de activación en las capas ocultas y en la capa de salida debe ser la función sigmoide.
- Las librerías utilizadas son **Scikit-learn**, **TensorFlow** y **Keras**.

4.2. Modelo A: Perceptrón

El Perceptrón es el clasificador lineal más simple: una única neurona de salida que aplica una función escalón (Heaviside) sobre la combinación lineal de las entradas. Su frontera de decisión es un hiperplano en el espacio de características. No tiene capas ocultas.

La implementación usa `sklearn.linear_model.Perceptron` con los siguientes parámetros:

- `fit_intercept=True`: garantiza el término de sesgo.
- `max_iter=1000`: número máximo de épocas de entrenamiento.
- `class_weight='balanced'`: compensa el desbalanceo de clases detectado en la Fase 1, escalando los pesos de las muestras inversamente proporcional a la frecuencia de cada clase.
- `random_state=42`: semilla para reproducibilidad.

Arquitectura: Entrada (82) → Neurona de salida (función escalón, *bias* incluido).

4.3. Modelo B: Red neuronal con una capa oculta

El Modelo B tiene una capa oculta con 82 neuronas, igual al número de variables de entrada obtenido tras el preprocesamiento de la Fase 1. La función sigmoide se aplica tanto en la capa oculta como en la de salida. Todas las neuronas incluyen sesgo (`use_bias=True`, valor por defecto en Keras).

Arquitectura:

Entrada(82) → Dense(82, σ) → Dense(1, σ)

La función sigmoide se define como:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Parámetros de entrenamiento:

- Optimizador: Adam (tasa de aprendizaje por defecto: 0,001).
- Función de pérdida: entropía cruzada binaria (`binary_crossentropy`).
- Épocas máximas: 50, con *early stopping* (paciencia 5 épocas, restaurando los mejores pesos).
- Tamaño de lote (*batch size*): 256.
- Validación interna: 10 % del conjunto de entrenamiento.

- Compensación de desbalanceo: `class_weight` calculado con `compute_class_weight('balanced')` en coherencia con el desbalanceo 76/24 documentado en la Fase 1.

4.4. Modelo C: Red neuronal con dos capas ocultas

El Modelo C agrega una segunda capa oculta, pero con un cuello de botella severo: ambas capas tienen exactamente 2 neuronas cada una. La función sigmoide se aplica en todas las capas y el sesgo está en cada neurona.

Arquitectura:

$$\text{Entrada}(82) \rightarrow \text{Dense}(2, \sigma) \rightarrow \text{Dense}(2, \sigma) \rightarrow \text{Dense}(1, \sigma)$$

Los parámetros de entrenamiento son idénticos a los del Modelo B.

La tabla 2 resume las tres arquitecturas implementadas.

Tabla 2: Resumen de arquitecturas implementadas

Modelo	Capas ocultas	Activación	Bias
A – Perceptrón	Ninguna	Escalón (Heaviside)	Sí
B – 1 capa oculta (82 n.)	1 capa $\times$ 82	Sigmoide	Sí
C – 2 capas ocultas (2 n. c/u)	2 capas $\times$ 2	Sigmoide	Sí

5. Análisis de resultados y evaluación

Todos los modelos se evaluaron sobre el conjunto de prueba (`X_test`), compuesto por 9.769 muestras: 7.431 de la clase $\leq 50K$ y 2.338 de la clase $> 50K$.

5.1. Modelo A: Perceptrón

5.1.1. Matriz de confusión

La tabla 3 presenta la matriz de confusión del Modelo A.

Tabla 3: Matriz de confusión – Modelo A (Perceptrón)

		Predicción	
		<=50K	>50K
Real	<=50K	6.954	477
	>50K	1.169	1.169

5.1.2. Métricas de desempeño

Tabla 4: Métricas de desempeño – Modelo A (Perceptrón)

Clase	Precisión	Recall	F1-score	Soporte
<=50K	0,86	0,94	0,89	7.431
>50K	0,71	0,50	0,59	2.338
<i>Exactitud (Accuracy)</i>		0,83		9.769
<i>Macro avg</i>	0,78	0,72	0,74	9.769
<i>Weighted avg</i>	0,82	0,83	0,82	9.769

Análisis: El Perceptrón tiene la mayor exactitud bruta (83 %), pero ese número resulta engañoso dado el desbalanceo documentado en la Fase 1 (76 % vs. 24 %). Un modelo que predijera siempre <=50K alcanzaría el 76 % sin aprender nada. El *recall* de apenas el 50 % para la clase >50K deja en evidencia el problema: uno de cada dos casos de altos ingresos queda mal clasificado. La causa es la linealidad inherente del modelo, que no puede capturar las relaciones no lineales entre variables como educación, ocupación y capital acumulado, ya evidenciadas en el análisis exploratorio de la Fase 1.

5.2. Modelo B: Red neuronal con una capa oculta

5.2.1. Matriz de confusión

Tabla 5: Matriz de confusión – Modelo B (1 capa oculta, 82 neuronas)

		Predicción	
		<=50K	>50K
Real	<=50K	5.932	1.499
	>50K	351	1.987

5.2.2. Métricas de desempeño

Tabla 6: Métricas de desempeño – Modelo B (1 capa oculta, 82 neuronas)

Clase	Precisión	Recall	F1-score	Soporte
<=50K	0,94	0,80	0,86	7.431
>50K	0,57	0,85	0,68	2.338
<i>Exactitud (Accuracy)</i>		0,81		9.769
<i>Macro avg</i>	0,75	0,82	0,77	9.769
<i>Weighted avg</i>	0,85	0,81	0,82	9.769

Análisis: El Modelo B es el que mejor identifica la clase minoritaria: *recall* de 85 % para >50K y F1 macro de 0,77, el más alto de los tres modelos. La capa oculta de 82 neuronas con activación sigmoide le permite aprender representaciones intermedias no lineales del espacio de entrada, compensando la limitación del Perceptrón. El uso de `class_weight='balanced'`, directamente motivado por el desbalanceo documentado en la Fase 1, es determinante para este resultado.

5.3. Modelo C: Red neuronal con dos capas ocultas

5.3.1. Matriz de confusión

Tabla 7: Matriz de confusión – Modelo C (2 capas ocultas, 2 neuronas c/u)

		Predicción	
		<=50K	>50K
Real	<=50K	5.828	1.603
	>50K	352	1.986

5.3.2. Métricas de desempeño

Tabla 8: Métricas de desempeño – Modelo C (2 capas ocultas, 2 neuronas c/u)

Clase	Precisión	Recall	F1-score	Soporte
<=50K	0,94	0,78	0,86	7.431
>50K	0,55	0,85	0,67	2.338
<i>Exactitud (Accuracy)</i>		0,80		9.769
<i>Macro avg</i>	0,75	0,82	0,76	9.769
<i>Weighted avg</i>	0,85	0,80	0,81	9.769

Análisis: A pesar del cuello de botella severo (82 variables comprimidas en 2 neuronas por capa), el Modelo C alcanza métricas muy cercanas al Modelo B en la clase minoritaria (*recall* = 85 %, *F1* = 0,67). No obstante, su exactitud global (80 %) y su *F1* para >50K (0,67) son ligeramente inferiores. Con solo 2 neuronas por capa, la red carece de capacidad representacional suficiente para el espacio de 82 dimensiones resultante del preprocesamiento de la Fase 1, lo que se evidencia en los 1.603 falsos positivos frente a los 1.499 del Modelo B.

6. Manejo del desbalanceo de clases

Como se documentó en la Fase 1, el dataset presenta un desbalanceo importante: 76,1 % en la clase <=50K y 23,9 % en la clase >50K. Este desbalanceo afecta tanto el entrenamiento

como la evaluación de los modelos y determinó la estrategia adoptada en esta fase:

- **Durante el entrenamiento:** Se usó `class_weight='balanced'` en el Perceptrón y `class_weight` calculado con `compute_class_weight('balanced')` en los Modelos B y C. Esto asigna mayor penalización a los errores sobre la clase minoritaria en la función de pérdida, forzando al modelo a no ignorar el >50K.
- **Durante la evaluación:** Se reportaron métricas por clase y los promedios macro y ponderado. La exactitud bruta no es indicador suficiente con datos desbalanceados; por eso el *recall* de >50K y la F1 macro son los indicadores principales de desempeño en este análisis.
- **Partición estratificada (heredada de la Fase 1):** La partición con `stratify=y` garantiza que el conjunto de prueba conserve la misma proporción de clases 76/24 que el conjunto de entrenamiento, evitando que las métricas de evaluación reflejen una distribución distinta a la real.

7. Análisis comparativo

7.1. Tabla consolidada de métricas

La tabla 9 consolida las métricas de los tres modelos.

Tabla 9: Comparación de métricas de desempeño entre los tres modelos

Modelo	Acc.	Prec. >50K	Rec. >50K	F1 >50K	F1 macro	F1 wgt.
A – Perceptrón	0,83	0,71	0,50	0,59	0,74	0,82
B – 1 capa oculta (82 n.)	0,81	0,57	0,85	0,68	0,77	0,82
C – 2 capas ocultas (2 n.)	0,80	0,55	0,85	0,67	0,76	0,81

7.2. Discusión técnica

7.2.1. Modelo A – Perceptrón

El Perceptrón tiene la mayor exactitud bruta (83 %), pero ese número engaña cuando el dataset está tan desbalanceado (76 % vs. 24 %), como se estableció en la Fase 1. Un *recall* del 50 % para la clase minoritaria deja en claro que uno de cada dos casos de altos ingresos queda mal clasificado. La causa es estructural: su frontera de decisión lineal no puede capturar las interacciones entre variables como educación, ocupación y capital acumulado, cuya relación no lineal con el ingreso ya era visible en el análisis exploratorio de la Fase 1.

7.2.2. Modelo B – Red neuronal con una capa oculta

El Modelo B es el mejor de los tres. La capa oculta de 82 neuronas (dimensión igual al número de variables de entrada tras el preprocesamiento de la Fase 1) con activación sigmoide le permite aprender representaciones intermedias no lineales. El optimizador Adam, junto con el ajuste de pesos de clase directamente motivado por el desbalanceo de la Fase 1, garantiza que el modelo aprenda bien de la clase minoritaria. Su F1 macro de 0,77 y *recall* del 85 % para >50K son los más altos, mostrando el mejor equilibrio entre clases.

7.2.3. Modelo C – Red neuronal con dos capas ocultas

El Modelo C es más profundo, pero el cuello de botella de 2 neuronas por capa limita severamente su capacidad representacional frente a un espacio de 82 dimensiones. Las métricas quedan muy cerca de las del Modelo B, lo que sugiere que el optimizador Adam y los pesos de clase compensan parcialmente la restricción. Aun así, la diferencia en F1 de la clase minoritaria (0,67 vs. 0,68) y en exactitud (80 % vs. 81 %) confirma que el cuello de botella tiene un costo real.

7.3. Justificación de la arquitectura ganadora

El **Modelo B** (red neuronal con una capa oculta de 82 neuronas) presenta el mejor desempeño para este problema por las siguientes razones:

1. **Mayor F1 macro (0,77):** Mejor equilibrio entre clases, condición necesaria dado el

desbalanceo 76/24 documentado en la Fase 1.

2. **Mayor F1 para la clase minoritaria (0,68):** Detecta más correctamente las personas con ingresos superiores a 50K, que es la clase de mayor interés predictivo.
3. **Capacidad representacional adecuada:** 82 neuronas en la capa oculta permiten aprender una representación rica del espacio de 82 dimensiones, sin el cuello de botella del Modelo C ni la rigidez lineal del Modelo A.
4. **Coherencia con la Fase 1:** El número de neuronas en la capa oculta (82) coincide con el número de variables de entrada obtenido tras el preprocesamiento, estableciendo una conexión directa y justificada con los resultados de esa fase.

8. Conclusiones

1. El **Modelo B** (red neuronal con una capa oculta de 82 neuronas) es la arquitectura con mejor desempeño para este problema: F1 macro de 0,77 y *recall* del 85 % para la clase minoritaria >50K.
2. El **Perceptrón** (Modelo A), aunque logra la mayor exactitud bruta (83 %), falla en detectar la mitad de los casos de ingresos superiores a 50K (*recall* = 50 %). Con datos desbalanceados y cuando la clase minoritaria es la más relevante, ese indicador basta para descartarlo como solución adecuada.
3. El **Modelo C**, con dos capas ocultas de 2 neuronas cada una, alcanza métricas sorprendentemente cercanas al Modelo B. Adam y los pesos de clase compensan parcialmente la restricción, pero la ligera inferioridad en todas las métricas confirma que el cuello de botella penaliza.
4. El desbalanceo de clases identificado en la Fase 1 fue determinante en la estrategia de modelado: sin `class_weight='balanced'`, los tres modelos habrían tendido a ignorar la clase minoritaria, produciendo modelos con alta exactitud bruta pero escasa utilidad práctica.
5. Las decisiones de preprocesamiento de la Fase 1 —especialmente las 82 variables resultantes y la normalización con `StandardScaler`— inciden directamente en la arquitectura y en la convergencia de los modelos basados en gradiente (B y C). Esta continuidad entre fases es la base del diseño experimental presentado en este informe.

9. Referencias

1. BACHE, K.; LICHMAN, M. *UCI Machine Learning Repository: Adult Data Set*. Irvine: University of California, School of Information and Computer Science, 1996. Disponible en: <https://archive.ics.uci.edu/ml/datasets/adult>
2. BISHOP, Christopher M. *Pattern Recognition and Machine Learning*. New York: Springer, 2006. ISBN 978-0-387-31073-2.
3. CHOLLET, François. *Deep Learning with Python*. 2a ed. Shelter Island: Manning Publications, 2021. ISBN 978-1617296864.
4. GÉRON, Aurélien. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3a ed. Sebastopol: O'Reilly Media, 2022. ISBN 978-1098125974.
5. INSTITUTO COLOMBIANO DE NORMAS TÉCNICAS Y CERTIFICACIÓN (ICONTEC). *NTC 1486: Documentación, presentación de tesis, trabajos de grado y otros trabajos de investigación*. 6a ed. Bogotá: ICONTEC, 2008.
6. PEDREGOSA, F. *et al.* Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825–2830, 2011.
7. TENSORFLOW TEAM. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. 2015. Disponible en: <https://www.tensorflow.org>